NovaShyld Internship Documentation — Task 4

Student Name: Ishal Iqbal

Task Name: Task 4 - Web Application Security Testing

Internship Program: NovaShyld Cybersecurity Internship

Target Application: DVWA (Damn Vulnerable Web Application)

GitHub Repository: https://github.com/ishaliqbal/NovaShyld-Task_4

Introduction & Task Objective

In modern web security, web applications are primary targets for cyber exploits due to public accessibility and direct connections to backend databases. The primary objective of NovaShyld Task 4: Web Application Security Testing was to set up a controlled web application lab environment, analyze application behavior against OWASP Top 10 vulnerabilities, and successfully exploit and document two major security flaws: SQL Injection (SQLi) and Reflected Cross-Site Scripting (XSS).

1. Lab Setup & Execution Environment

The lab testing environment was constructed using VirtualBox running a Kali Linux virtual machine instance. The target application used was DVWA (Damn Vulnerable Web Application) hosted locally on Apache2 and MySQL servers at http://127.0.0.1:42001.

- **Security Level Setting:** Configured DVWA Security setting to **Low** to examine unmitigated code vulnerabilities.
- **Testing Flow:** Intercepted and analyzed HTTP POST/GET parameter behaviors directly in the browser and Burp Suite Community Edition.

2. Vulnerability Findings & Proof of Concept (PoC)

2.1 Finding 1: SQL Injection (SQLi) — OWASP A03:2021

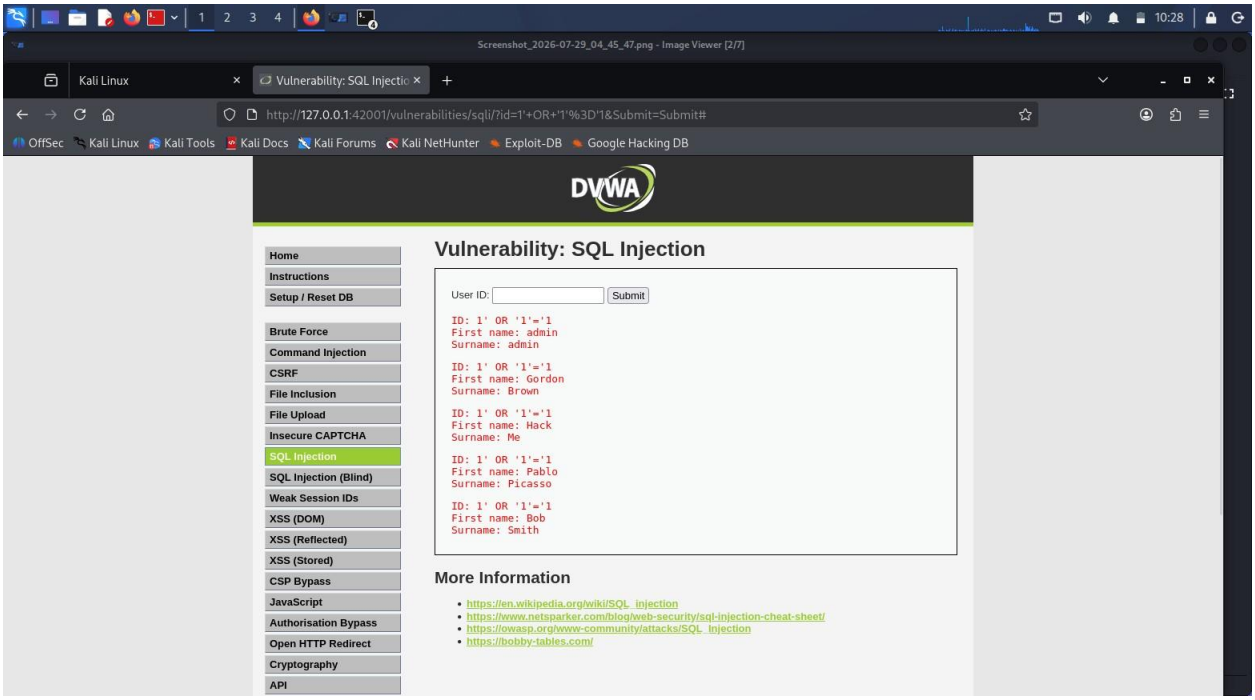
SQL Injection occurs when user input is directly concatenated into a database query string without proper sanitization or parameterization.

- **Tested Section:** SQL Injection tab in DVWA
- **Input Parameter:** User ID input box
- **Injected Payload:** 1' OR '1'='1 **Backend Query Logic Breakdown:**

- **Original Query:** SELECT first_name, last_name FROM users WHERE user_id = '$id';
- **Injected Query:** SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1';

Technical Impact & Result:

The single quote ' closed the original parameter string, while OR '1'='1' created a logical condition that always evaluates to TRUE. The database engine bypassed single-user lookup and dumped all user account credentials (including admin details) onto the web page.



2.2 Finding 2: Reflected Cross-Site Scripting (XSS) — OWASP A03:2021

Reflected XSS occurs when an application receives user input in an HTTP request and includes that input in the immediate HTML response without sanitization or escaping.

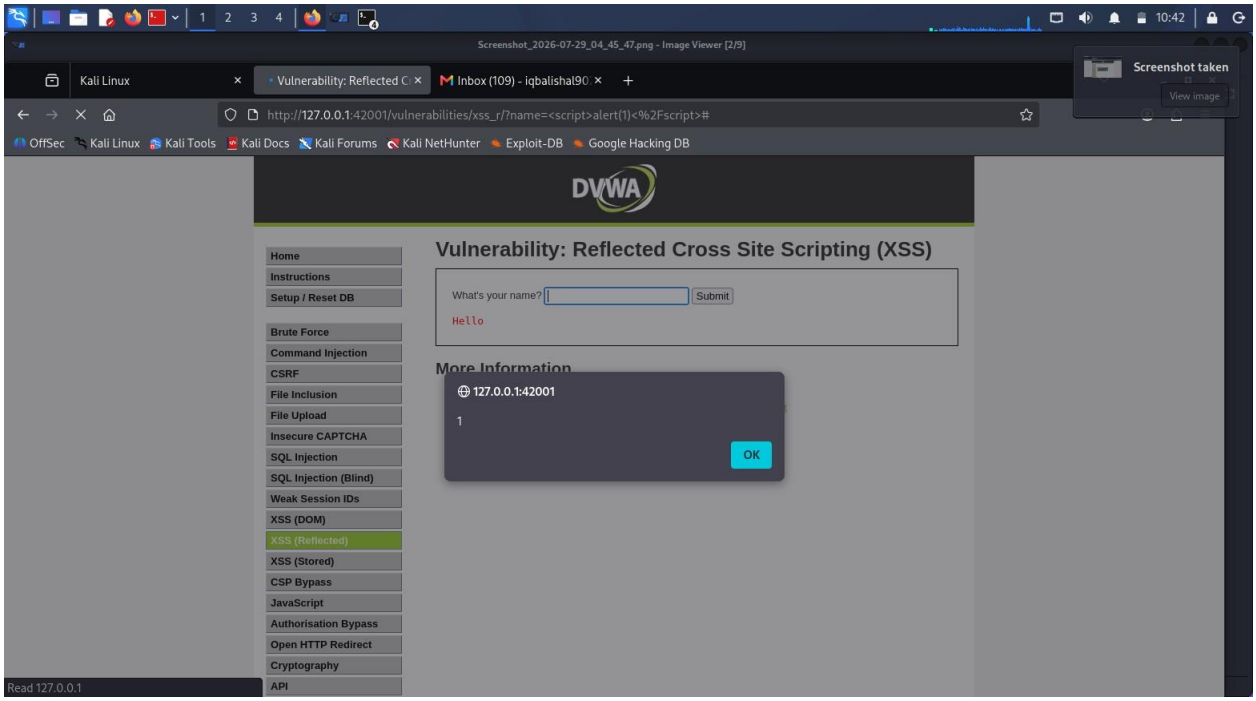
- **Tested Section:** XSS (Reflected) tab in DVWA
- **Input Parameter:** "What's your name?" input field
- **Injected Payload:** <script>alert(1)</script>

Client Execution Mechanics:

1. **Input:** Submitted via HTTP GET request (?name=<script>alert(1)</script>)

2. **Response:** Application rendered script directly into DOM: Hello <script>alert(1)</script>
3. **Browser:** Executed script tag, rendering a JavaScript dialog box displaying '1' **Technical Impact & Result:**

The browser treated the injected text as executable JavaScript rather than plain text, rendering a pop-up alert dialog displaying 127.0.0.1:42001 says: 1. This confirmed that an attacker could execute arbitrary scripts, steal session cookies, or perform actions on behalf of authenticated users.



3. Summary Finding Matrix

Vulnerabi	OWASP Benchma	Target Paramet	Severi	Status
SQL Inject (SQLi)	A03:2021 Injection	User ID (GET/POS	Critica	Exploited & Captured

Vulnerabi	OWASP Benchma	Target Paramete	Severi	Status
Reflected	A03:2021 Injection	name Paramete	High	Exploited & Captured

4. Security Remediation & Fix Recommendations

1. How to Fix SQL Injection:

Developers must use **Parameterized Queries (Prepared Statements)** instead of concatenating raw user strings into SQL code. Prepared statements ensure the database engine strictly treats user input as data, not executable commands.

2. How to Fix Reflected XSS:

Developers must implement strict **Context-Aware Output Encoding** (e.g., converting < to < and > to >) before reflecting user input in the browser. Additionally, enforcing a robust **Content Security Policy (CSP)** header helps block unauthorized inline script execution.